

Operating Systems

`fork()`: additional details and corner cases

Mirco Marchetti, Riccardo Lancellotti

SECloud research group
University of Modena and Reggio Emilia

AY 2025/26

Why this deck of slides?

- We explored the basic behavior of `fork()`, but neglected some details
- We will now go deeper to better understand a few behaviours of `fork()` that are counterintuitive and may be surprising
- All code examples are on Moodle! Esercitazioni e dimostrazioni → Ancora fork. Altri esempi e casi particolari

Parent and child do not share memory!

- 01-variabili.c
 - the parent declares a variable before fork, two child use the “same” variable
 - the variable is **not shared**!
 - the **name** is the same, but it lives in the **stack** and each process has its own copy of the **stack**!
- 02-malloc.c
 - the parent allocates a memory buffer before fork, two child use the “same” buffer
 - the memory buffer is **not shared**!
 - the **name** is the same, the **value of the pointer** is the same, but it lives in the **heap** and each process has its own copy of the **heap**!

Same pointer point to different memory?

- Yes, two processes have identical pointers, but they point to different memory areas. This is very confusing...
- Each process have memory addresses that are **relative** to the memory page(s) assigned to the process by the OS
- Different processes have different memory pages (child pages are copied from the parent) → the same relative addresses point to different “physical” addresses in memory
- `02b-malloc-pfn.c`
 - this program prints the **physical frame number** (PFN) of the memory page assigned to the process, and the offset of the pointer within that page
 - where to find it? In the `proc` filesystem, of course! `/proc/<pid>/pagemap`
 - **Note:** this has to be run as root, since the `pagemap` file is not readable by normal users
 - for more info: `man proc_pid_pagemap`

Parent and child share files!

- `03-file-write.c`
 - the parent opens a file before fork, two child write to the file using the same file descriptor
- `04-file-read.c`
 - the parent opens a file before fork, two child read from the file using the same file descriptor
 - the file descriptor is **shared!**
- `05-file-seek.c`
 - the parent opens a file before fork, the child changes the current position in the file with `lseek()`, the parent reads from the file using the same file descriptor
 - the file descriptor is **shared!**
- the file descriptor is **shared!**
 - the **name** is the same, the **value of the file descriptor** is the same, and it points to the **same** open file description (i.e., the same file offset, the same file status flags, etc.)
 - we will better understand what an open file description is later in the course...

And if the file is opened after the fork?

- `06-file-after-fork.c`
 - parent and child open the same file **after** fork
 - the file descriptor is **not shared!**
 - they are just two different processes opening the same file, each with its own file descriptor

Weird behaviors: buffering and process termination...

- `07-buffered-io.c`
 - parent's buffer are filled with data before forking twice (call to `printf` without terminating `\n`)
 - all child terminate by calling `return 0`
 - buffers are flushed 4 times!
- Keep in mind that also buffers are copied by fork, so they are not shared, but they are identical at the moment of the fork. This can lead to very weird behaviors if you are not careful!

return, exit() or _exit()?

- `08-return-exit.c`
 - returning from a function does not terminate the process
 - `exit()` terminates the process
 - returning from `main()` is equivalent to calling `exit()` with the return value as argument

- 09-exit-_exit.c
 - `exit()` performs some cleanup operations before terminating the process, such as flushing the standard I/O buffers, closing open file descriptors, etc.
 - `_exit()` terminates the process immediately without performing any cleanup operations
- 10-fork-exit-_exit.c
 - if you call `exit()` in a child process, the buffers will be flushed twice (once in the child and once in the parent), which can lead to unexpected behaviors if the buffers contain data that is not yet written to the file or the terminal. In this case, it is better to call `_exit()` in the child process to avoid flushing the buffers twice!

- It is possible to define cleanup handlers that will be called when the process terminates using `atexit()`
- `11-atexit.c`
 - `exit()` will call the cleanup handlers registered with `atexit()`
 - `_exit()` will **not** call them
- `12-fork-atexit.c`
 - if you call `exit()` in a child process, the cleanup handlers registered with `atexit()` will be also called in the child process